

Scope statement:
Smart Building Emergency

Authors:

BESBES Ines – BEN SOUISSI Rihem – HAMROUNI Nesrine

Faculty Adviser:

Dr. Eng. Mohamed Becha Kaaniche

Contents

1 Concept **1**
1.1 Project Context . . . 1
1.2 Problem Statement . . . 1
1.3 Objectives . . . 1

2 Client **2**
2.1 Client Description . . . 2
2.2 Target Population . . . 2

3 Functional need **3**
3.1 Sensor Monitoring and IoT Communication . . . 3
3.2 Emergency Detection and Incident Management . . . 3
3.3 AI-Powered Emergency Analysis . . . 3
3.4 Location-Based Services and Notifications . . . 3
3.5 Client Progressive Web Application . . . 3
3.6 Administrator Progressive Web Application . . . 3

4 AI Agent and LLMOps Integration **3**
4.1 Purpose of the AI Agent . . . 3
4.2 AI Agent Architecture . . . 4
4.3 Emergency Analysis Workflow . . . 4
4.4 Risk Assessment and Response Recommendations . . . 5
4.5 LLMOps Integration . . . 5
4.6 Expected Outcome . . . 5

5 Equipment **6**

6 Technologies choice **6**
6.1 Back-End . . . 6
6.2 Middleware . . . 6
6.3 Front-End . . . 7
6.4 IoT integration . . . 7

7 Architecture **7**

8 Timeline and tasks **8**

9 Methodology **8**
9.1 Development Methodology: Extreme Programming (XP) . . . 8
9.2 Extreme Programming Values . . . 8
9.3 The Extreme Programming Cycle . . . 9
9.4 Application of XP to the Project . . . 10

10 Limitations **11**
10.1 Hardware and Sensor Limitations 11
10.2 Network and Communication Limitations 12
10.3 AI Agent Limitations 12
10.4 Software Integration Limitations 12
10.5 Security and Privacy Limitations 13

11 Business Study **13**
11.1 Business Model Canvas (BMC) 13
11.2 SWOT Analysis 15
11.3 Business Study Conclusion 15

12 Deliverables **15**
12.1 Expected Final Outcome 17

13 Conclusion **18**

1 Concept

1.1 Project Context

The increasing adoption of Internet of Things (IoT) technologies and artificial intelligence has created new opportunities to improve safety and monitoring in modern buildings. Smart buildings can integrate connected sensors, cameras, communication networks, and intelligent software to monitor environmental conditions and respond to abnormal situations.

Smart Building Emergency is a Cloud of Things platform designed to improve emergency detection and response within a smart building. The system combines IoT sensors, optional camera data, location information, occupancy information, historical measurements, and an AI agent to support intelligent emergency management.

1.2 Problem Statement

Emergency management in buildings requires more than collecting environmental measurements. A useful system must interpret incoming data, identify the affected location, evaluate the potential danger, determine whether people may be exposed, and communicate appropriate information to the relevant users.

A simple threshold-based system may generate an alert when a sensor exceeds a predefined value, but it may not adequately consider the evolution of the measurements, occupancy, historical data, camera observations, or the precise location of the event.

These challenges lead to the following central problem:

> **How can an intelligent, IoT-based platform detect potential emergencies in a smart building, assess their severity using contextual and multimodal information, identify affected areas and occupants, and coordinate an appropriate response in real time?**

1.3 Objectives

1.3.1 General Objective

The main objective is to design and develop a Cloud of Things platform that improves smart building safety by combining real-time IoT monitoring, contextual emergency detection, AI-assisted risk assessment, location-based services, and coordinated notifications.

1.3.2 Specific Objectives

The project has the following specific objectives:

1. **Real-time monitoring:** Collect and process environmental measurements.
2. **Emergency detection:** Identify potential fire and gas leak scenarios using sensor measurements, their evolution over time, and additional information when available.
3. **Contextual risk assessment:** Analyze sensor readings, occupancy information, historical measurements, and optional camera observations.
4. **AI-assisted decision-making:** Integrate an AI agent capable of retrieving information, invoking authorized tools, evaluating situations, and explaining its conclusions.

5. **Incident management:** Create and track emergency incidents, maintain their status, and provide administrators with relevant incident information.
6. **Targeted notifications:** Inform administrators and potentially affected occupants about detected incidents and provide appropriate safety recommendations.
7. **Location-Based Services:** Display incidents on a building map or floor plan and identify affected safety zones and nearby users when the necessary location data is available.
8. **User application development:** Provide a client PWA for occupants and a separate administrator PWA for monitoring, incident management, device management, and simulation control.
9. **IoT-cloud integration:** Establish communication between IoT devices and the cloud backend through MQTT/MQTTS and HiveMQ.
10. **AI observability:** Implement LLMOps mechanisms to record and monitor agent requests, tool calls, decisions, response times, token usage, errors, and results.

2 Client

2.1 Client Description

The client of the **Smart Building Emergency** project is the organization responsible for managing and ensuring the safety of a smart building. This may be a company, a university, a residential building manager, or another organization operating premises equipped with connected monitoring devices.

2.2 Target Population

The platform is intended for the following categories of users:

2.2.1 Building Occupants

Building occupants include employees, students, residents, visitors, and other people present inside the building.

2.2.2 Building Administrators

Administrators are responsible for monitoring the building's safety status and managing the platform.

2.2.3 Technical and Maintenance Staff

Technical staff are responsible for maintaining the IoT infrastructure and ensuring that connected devices and software components operate correctly. Depending on the implemented features and assigned permissions, they may inspect sensor measurements, identify communication problems, verify device status, and support system maintenance.

3 Functional need

3.1 Sensor Monitoring and IoT Communication

The system shall collect temperature, humidity, gas, and smoke measurements from connected sensors and transmit them to the backend via MQTT using the HiveMQ broker. Each sensor shall have a unique identifier associated with its building, floor, room, or safety zone.

3.2 Emergency Detection and Incident Management

The system shall detect potential fire and gas leak situations using sensor measurements and configured rules. It shall assess emergency severity based on sensor trends, location, occupancy, and historical data, while storing relevant measurements and incident records in MongoDB for consultation and historical analysis.

3.3 AI-Powered Emergency Analysis

The system shall integrate an AI agent using LangChain4j-CDI to analyze emergencies contextually. Through authorized tools, the agent shall retrieve available sensor data, location, occupancy, and historical measurements, then explain its assessment based on relevant observations and contextual information.

3.4 Location-Based Services and Notifications

The system shall associate each incident with the location of its originating sensor or affected zone and notify relevant users according to their roles, configured notification rules, and available contact or location information.

3.5 Client Progressive Web Application

The Client PWA shall display the building's safety status and active incidents, allowing users to consult incident details, including type, location, status, and available safety recommendations.

3.6 Administrator Progressive Web Application

The Admin PWA shall provide a centralized dashboard to monitor sensor data, active incidents, and platform status. It shall allow authorized administrators to manage registered sensors and their locations, consult incident details and history, and perform supported incident status-management operations.

4 AI Agent and LLMOps Integration

4.1 Purpose of the AI Agent

The Smart Building Emergency platform integrates an AI agent using **LangChain4j-CDI** to support emergency analysis and decision-making. The agent processes available sensor measurements and retrieves contextual information to assess potential risks, explain its conclusions, and recommend appropriate responses.

4.2 AI Agent Architecture

The AI agent is integrated into the backend developed with Jakarta EE 11 and deployed on WildFly. LangChain4j-CDI provides the integration between the application and the selected language model.

The agent interacts with backend services through authorized tools. Relevant data is retrieved from MongoDB or other available data sources, while sensor measurements are received through the MQTT communication pipeline.

The main components are:

- **Language Model (LLM):** Interprets the available information and generates assessments, explanations, and recommendations.
- **Agent Orchestration:** Coordinates the reasoning process and determines which authorized tools are required.
- **Tool Layer:** Exposes predefined backend operations to the agent.
- **Context and Data Layer:** Provides sensor measurements, location information, occupancy data, and historical records when available.
- **Incident Management Layer:** Handles incident creation, status updates, and supported notification operations.
- **LLMOps Layer:** Records relevant operational information and evaluates the agent's performance.

4.3 Emergency Analysis Workflow

The AI-assisted emergency workflow follows these steps:

1. **Event reception:** The backend receives a sensor measurement or a detection event.
2. **Initial detection:** Configured detection rules identify a potentially abnormal situation.
3. **Context retrieval:** The agent retrieves relevant sensor measurements, location information, occupancy data, and historical records when available.
4. **Risk assessment:** The agent evaluates the available evidence and determines a proposed risk level.
5. **Decision explanation:** The agent generates a human-readable explanation identifying the relevant observations and any important information gaps.
6. **Incident management:** The backend creates or updates an incident according to the configured rules and authorized operations.
7. **Notification and recommendation:** The platform sends configured alerts and provides relevant safety recommendations.
8. **Logging and evaluation:** The system records the available operational details for monitoring and later evaluation.

This workflow combines deterministic detection rules with AI-assisted contextual analysis. The language model supports interpretation and recommendations, while critical application operations remain controlled by backend logic.

4.4 Risk Assessment and Response Recommendations

The AI agent shall analyze the information available for each potential emergency and produce a structured assessment. Depending on the implemented design, the result may include:

- Incident category, such as a potential fire or gas leak.
- A proposed risk level, such as low, medium, or high.
- The affected building area, when location information is available.
- Relevant sensor observations and measurement trends.

The risk levels and associated response rules must be defined during implementation and validated using test scenarios. AI-generated conclusions must not be treated as proof that a building is safe.

4.5 LLMOps Integration

LLMOps refers to the practices used to monitor, evaluate, and improve the operation of the language-model component throughout development and testing.

The platform shall record relevant information about AI operations, subject to privacy and security constraints.

The Admin PWA should provide access to the available monitoring information so that administrators or developers can inspect agent behavior, identify failures, and compare results across test scenarios.

Sensitive information, credentials, and unnecessary personal data must not be included in operational logs.

4.6 Expected Outcome

The expected outcome is an AI-assisted emergency analysis component integrated into the Smart Building Emergency platform. It should retrieve relevant contextual information, assess potential incidents, explain its conclusions, invoke authorized backend tools, and provide response recommendations.

The LLMOps component should make the agent's behavior observable through operational logs, performance metrics, and scenario-based evaluation.

The implementation will prioritize the core workflow of sensor data retrieval, contextual analysis, incident creation, and recommendation generation. Advanced camera analysis, physical actuator control, and extensive monitoring dashboards will depend on technical feasibility and equipment availability.

5 Equipment

After conducting research on component availability, precision, and integration compatibility with Jakarta EE and MQTT protocols, the following equipment has been selected:

- **MQ-2 Gas and Smoke Sensor:** For detecting smoke and combustible gases to help identify potential fire hazards.
- **DHT22 (AM2302) Temperature and Humidity Sensor:** For monitoring environmental conditions and identifying abnormal temperature variations.
- **Passive Infrared (PIR) Occupancy Sensor / HC-SR501:** Detects infrared heat signatures emitted by human bodies within room perimeters. It provides dynamic occupancy presence data, allowing the AI agent to evaluate life hazard severity in the affected zone.
- **Raspberry Pi 4 Model B (4GB RAM):** Serves as the IoT gateway to collect sensor data and transmit it to the cloud platform through an MQTT broker.
- **MicroSD Card (Minimum 32 GB):** Provides high-speed storage for hosting the Raspberry Pi operating system (Raspberry Pi OS Lite), Python edge runtimes, MQTT client libraries, and local failure buffer logs.
- **ESP32-CAM (optional):** For capturing images or video to support visual verification of potential emergencies.
- **Prototyping Accessories:** Solderless breadboards, male-to-male and male-to-female jumper wires, stabilized $5V/3A$ USB-C power supplies, and local Wi-Fi router connectivity.

6 Technologies choice

6.1 Back-End

- **MongoDB:** NoSQL, document-oriented database used to store sensor measurements, building zones, emergency incidents, alerts, and historical data. Its flexible document structure is suitable for handling data collected from multiple IoT devices.
- **HiveMQ:** Widely used MQTT broker, serving as an intermediary for efficient and reliable messaging between IoT devices and the middleware and is available on the cloud.

6.2 Middleware

- **JAX-RS:** Java API for RESTful Web Services is a Java programming interface for creating web services with a REST architecture.
- **Jakarta Websocket:** Jakarta WebSocket provides an API for bidirectional, real-time communication between the server and connected clients. It will be used to transmit sensor updates, emergency alerts, and incident status changes to the user interface.
- **WildFly:** WildFly is an open-source Java application server used to deploy and execute the Jakarta EE backend services. It will host the REST API and the components responsible for processing incoming IoT data and coordinating emergency management operations.

- **LangChain4j-CDI:** LangChain4j provides Java integration with Large Language Models, while CDI facilitates their integration into Jakarta EE applications. In this project, it will support the implementation of an AI agent responsible for analyzing emergency context, interpreting collected information, and assisting with incident assessment and response recommendations.

6.3 Front-End

- **PWA (Progressive Web App):** A PWA is a cross-platform web application that provides a native-like experience to users. It allows you to develop applications for multiple platforms using web technologies like HTML, CSS, and JavaScript. PWAs are easy and quick to code and do not require prior web development knowledge.

6.4 IoT integration

- **Node-RED:** Node-RED is an open-source, flow-based programming tool widely used for IoT integration. It will be used to connect sensor data sources, perform basic preprocessing when necessary, and publish environmental measurements or simulated sensor readings to the MQTT broker. It can also help integrate different devices and communication services.
- **MQTT/MQTTS:** MQTT is a lightweight messaging protocol designed for communication between IoT devices and backend services. MQTTS adds TLS encryption to protect communication. In this project, MQTT/MQTTS will transmit sensor measurements and device status information through the HiveMQ broker.

7 Architecture

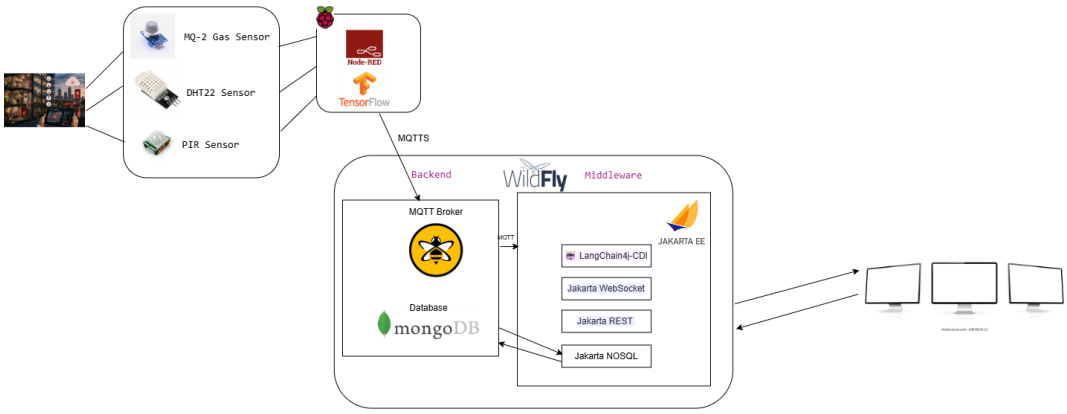


Figure 1: Architecture

The diagram above describes the main architecture of the emergency detection and response system which is mainly composed of IoT Edge, Backend, Middleware, AI Agent, and Frontend.

8 Timeline and tasks

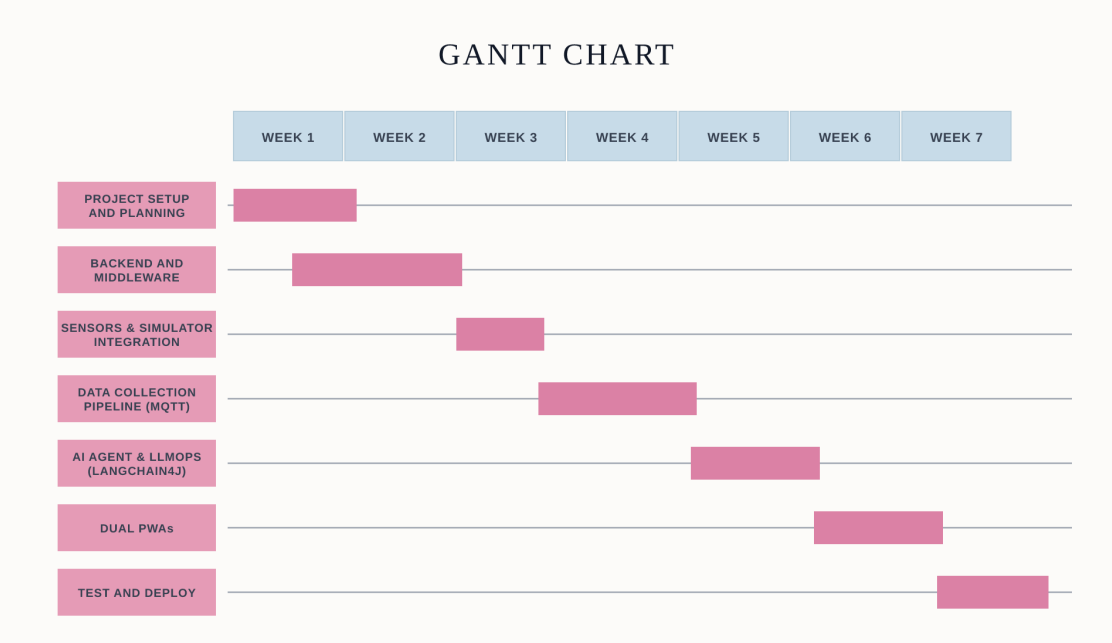


Figure 2: Gantt Diagram.

The Gantt chart in figure 2 illustrates the project timeline, showing the main tasks and their duration throughout the seven-week period. It highlights key tasks and ensures that each phase from setup and sensor integration to middleware development, machine learning, PWA, and final deployment is completed within the planned timeframe.

9 Methodology

9.1 Development Methodology: Extreme Programming (XP)

Extreme Programming (XP) is an Agile software development methodology that emphasizes simplicity, continuous feedback, collaboration, frequent testing, and adaptability to changing requirements.

XP is suitable for the **Smart Building Emergency** project because it involves several interconnected components, including IoT devices, cloud communication, backend services, an AI agent, and web applications. Developing these components incrementally allows the team to identify integration problems early and validate the main functionalities throughout the seven-week project.

The methodology focuses on delivering a functional system progressively rather than waiting until the end of development to integrate all components.

9.2 Extreme Programming Values

XP is based on five fundamental values:

Value	Application to the Project
Communication	Encourages continuous communication between team members and stakeholders. The team coordinates IoT integration, backend development, AI analysis, and frontend development to ensure compatibility between components.
Simplicity	Focuses on implementing the simplest solution that satisfies current requirements. The project prioritizes essential functionalities, such as sensor monitoring, emergency detection, incident management, and AI-assisted analysis.
Feedback	Uses testing and feedback to improve the system continuously. Sensor readings, MQTT communication, incident detection, and AI-generated recommendations are validated through regular tests.
Courage	Encourages developers to improve the code, address problems, and adapt to changes. The team corrects integration errors and revises detection rules when test results reveal weaknesses.
Respect	Promotes collaboration and respect for each team member's contributions. Tasks are coordinated according to responsibilities to ensure that the IoT, backend, AI, and frontend components work together.

9.3 The Extreme Programming Cycle

The XP development cycle consists of several iterative phases. Each iteration aims to deliver and validate a small part of the system.

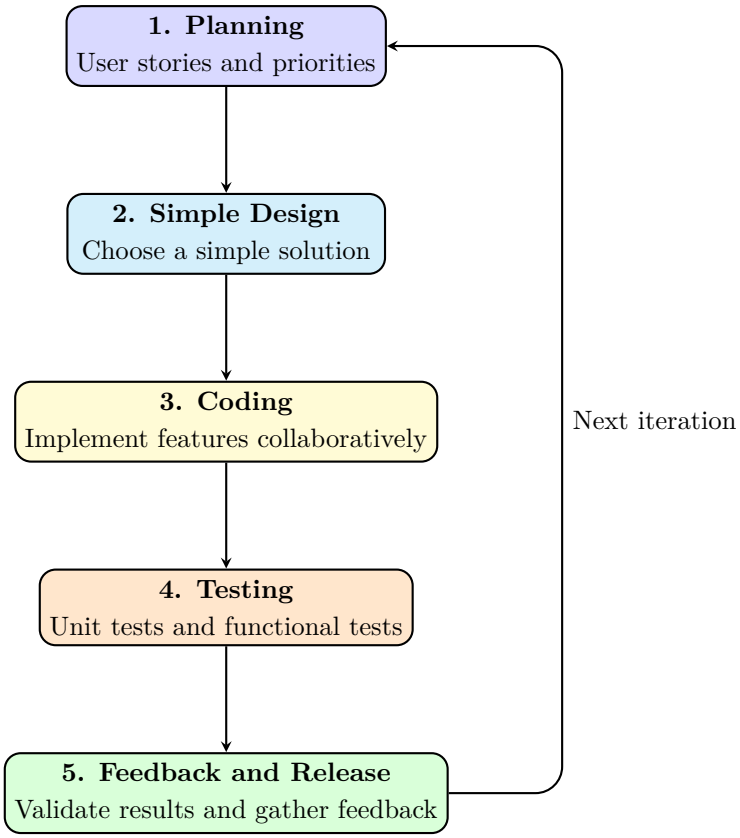


Figure 1: Extreme Programming (XP) Iterative Development Cycle

These phases are repeated until the project objectives are achieved or the available development period ends.

9.4 Application of XP to the Project

The XP cycle can be applied through the following development iterations:

Iteration	Main Activities	Expected Outcome
IoT Integration	Configure sensors or simulated data sources, Node-RED, MQTT, and HiveMQ.	Environmental measurements are transmitted to the platform.
Backend Development	Implement REST services, MongoDB storage, sensor data processing, detection rules, and incident management.	The backend can process measurements and manage incidents.
AI Integration	Integrate the LangChain4j-CDI agent and authorized tools for contextual information retrieval and risk assessment.	The agent produces contextual explanations and response recommendations.
Frontend Development	Develop the client and administrator PWAs and integrate them with the backend.	Users can consult safety information, while administrators can monitor sensors and manage incidents.

Testing and Evaluation	Test fire and gas leak scenarios, verify incident workflows, evaluate AI responses, and inspect operational logs.	An integrated prototype is validated against the main functional requirements.

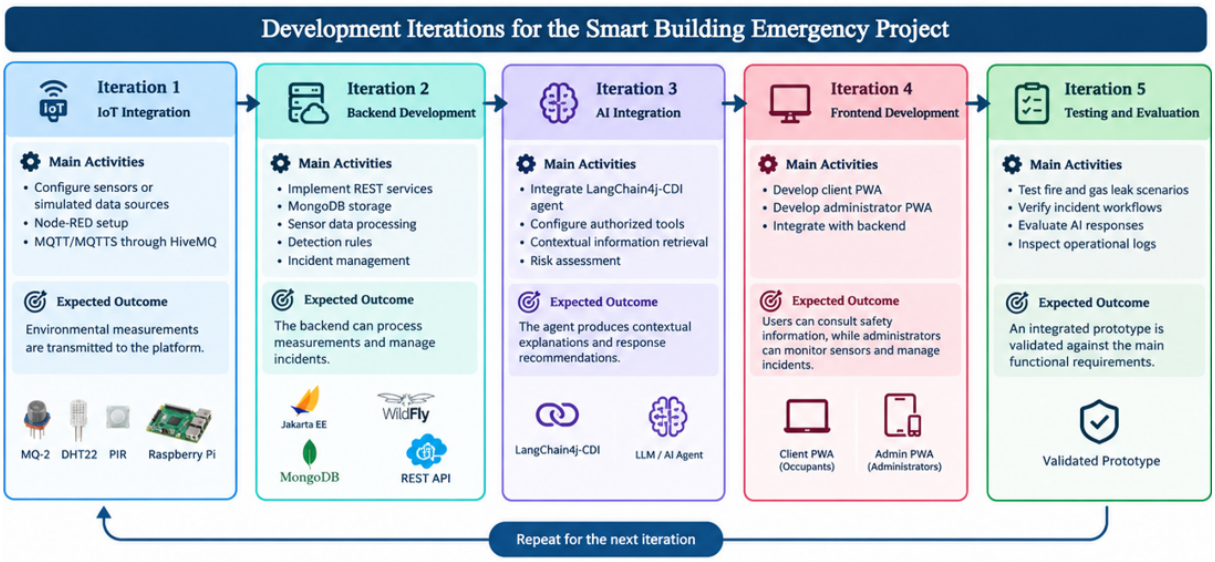


Figure 2: The XP cycle applied to our project

These iterations represent a proposed application of XP to the seven-week project schedule. Their duration and implementation scope may be adjusted according to technical constraints and equipment availability.

10 Limitations

Although Smart Building Emergency aims to improve building safety through IoT and AI, several technical and operational limitations may affect its implementation and performance.

10.1 Hardware and Sensor Limitations

The system depends on the availability and reliability of physical sensors, including the MQ-2 gas and smoke sensor, DHT22 temperature and humidity sensor, and PIR occupancy sensor.

- Sensor measurements may be affected by environmental conditions, calibration, and hardware quality.
- The MQ-2 provides indicative gas and smoke measurements and cannot, by itself, guarantee accurate identification of a specific hazard.
- The PIR sensor detects movement based on infrared changes and may not reliably detect a stationary person.

- Optional camera integration depends on the availability of the ESP32-CAM and suitable processing capabilities.

10.2 Network and Communication Limitations

The system relies on MQTT/MQTTS communication through HiveMQ and network connectivity between IoT devices and the backend.

- Network interruptions may delay measurements and emergency alerts.
- Internet or broker unavailability may temporarily prevent cloud communication.
- Reliable operation requires appropriate authentication, access control, and secure communication.
- Local buffering and recovery mechanisms may be needed to reduce data loss during connectivity failures.

10.3 AI Agent Limitations

The AI agent uses available measurements and contextual information to support emergency analysis and generate recommendations.

- The quality of its conclusions depends on the accuracy, completeness, and availability of input data.
- A language model may generate incorrect or misleading explanations.
- Risk assessments may be less reliable when historical measurements, occupancy information, or location data are unavailable.
- AI-generated recommendations must be evaluated against predefined scenarios and safety requirements.

Consequently, the AI agent should support decision-making rather than replace validated detection rules, emergency procedures, or human judgment.

10.4 Software Integration Limitations

The platform combines several technologies, including Jakarta EE, WildFly, MongoDB, HiveMQ, Node-RED, LangChain4j-CDI, and Progressive Web Applications.

- Integrating these technologies may require additional configuration and debugging.
- Compatibility between software versions and external services may introduce development difficulties.
- Real-time updates require reliable communication between the backend and user interfaces.
- AI integration depends on the selected language model, its availability, response time, and resource requirements.

10.5 Security and Privacy Limitations

The system may process sensitive information related to building occupancy, incident locations, and camera observations.

- Access to administrative functions and incident information must be restricted according to user roles.
- Personal information and sensitive credentials must not be unnecessarily stored in operational logs.
- Secure communication and appropriate data-retention policies are necessary.
- Compliance with applicable privacy and security requirements must be considered during implementation.

11 Business Study

11.1 Business Model Canvas (BMC)

The Business Model Canvas is a strategic tool used to describe how a project creates, delivers, and captures value. For Smart Building Emergency, it helps identify target customers, the value provided by the platform, the resources required, and potential sources of revenue.

The proposed business model focuses on organizations responsible for building safety, including companies, universities, residential building managers, and other smart-building operators.

BMC Component	Description
1. Customer Segments	Companies, universities, residential building managers, commercial building operators, facility management companies, and organizations responsible for building safety.
2. Value Propositions	Real-time environmental monitoring; early identification of potential fire and gas leak situations; AI-assisted risk assessment; location-based incident management; targeted notifications; and centralized monitoring through web applications.
3. Channels	Direct sales to building managers and organizations; partnerships with smart-building integrators; facility management providers; and online demonstrations of the platform.
4. Customer Relationships	Technical support, system configuration, onboarding, maintenance services, periodic software updates, and assistance with integrating sensors into existing building infrastructure.
5. Revenue Streams	Potential subscription fees for cloud-based monitoring; installation and configuration services; maintenance contracts; and premium features such as advanced analytics or integration with existing building management systems.

6. Key Resources	IoT sensors, Raspberry Pi gateway, cloud infrastructure, MQTT broker, backend services, MongoDB database, AI agent, frontend applications, software development expertise, and historical measurement data.
7. Key Activities	Sensor integration; real-time data collection; emergency detection; AI-assisted contextual analysis; incident management; notification delivery; software maintenance; testing; and AI performance monitoring.
8. Key Partnerships	IoT hardware suppliers, cloud service providers, MQTT infrastructure providers, smart-building integrators, facility management companies, and providers of AI models or services.
9. Cost Structure	Sensor and gateway acquisition; installation; cloud hosting; database and messaging infrastructure; AI model usage; software development; maintenance; cybersecurity; and testing.

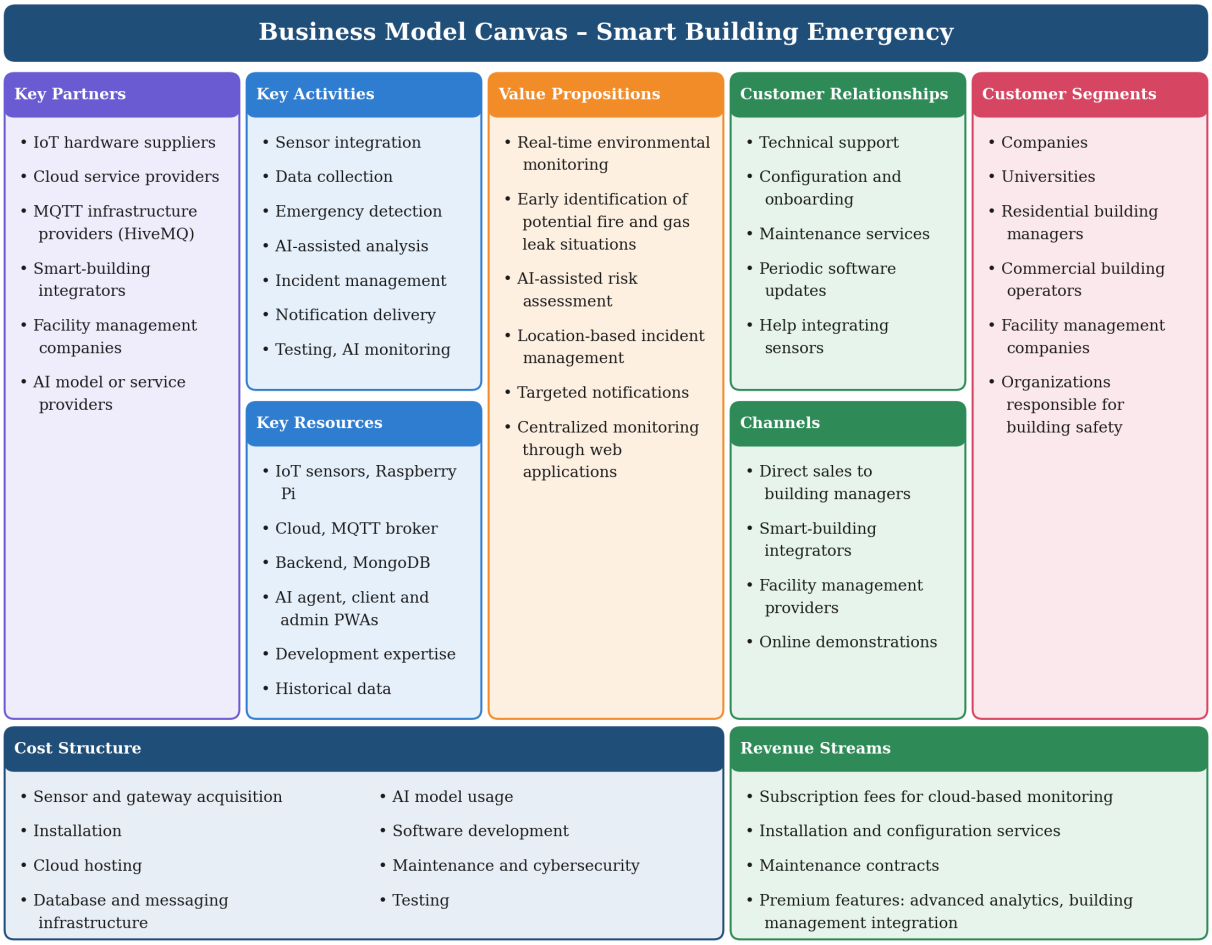


Figure 3: Business Model Canvas

The revenue streams and commercial partnerships described above are proposed possibilities

rather than confirmed existing revenues or agreements.

11.2 SWOT Analysis

SWOT analysis identifies the project's internal strengths and weaknesses and the external opportunities and threats that may influence its development and adoption.

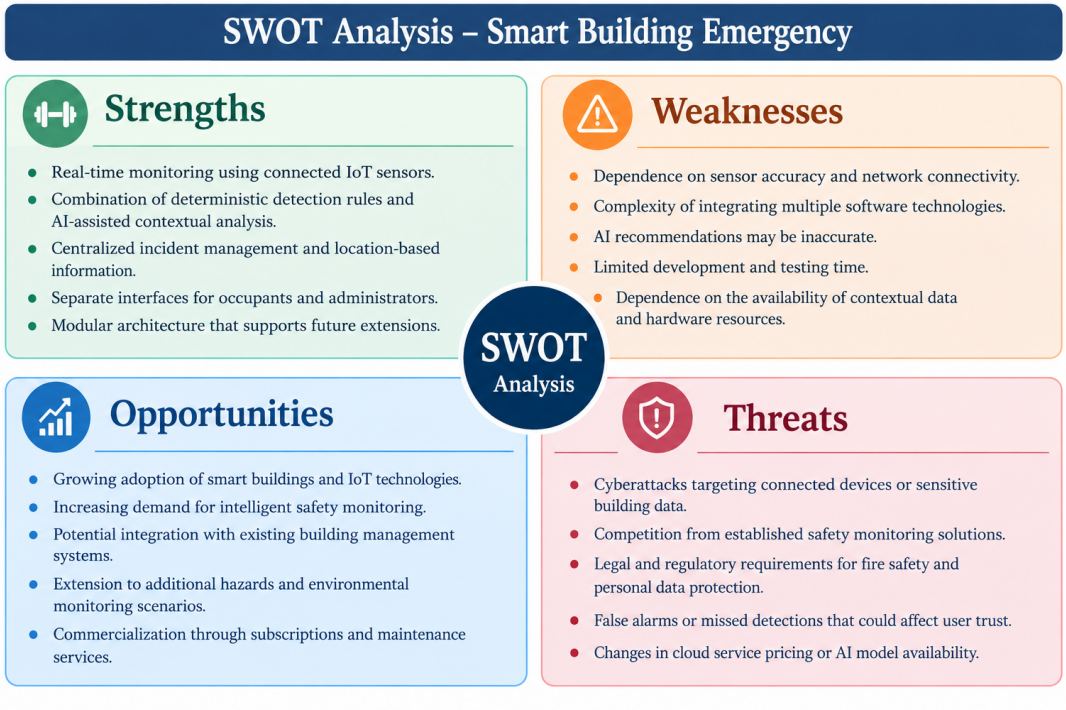


Figure 4: SWOT Analysis

11.3 Business Study Conclusion

The business study indicates that Smart Building Emergency has potential value for organizations seeking to improve building monitoring and emergency coordination. Its main differentiator is the combination of IoT measurements, contextual AI-assisted analysis, location-based services, and incident management within a single platform.

However, successful adoption depends on demonstrating reliable detection, protecting sensitive information, controlling operational costs, and validating the system under realistic conditions. A practical commercialization strategy would begin with a limited pilot deployment before considering larger-scale adoption.

12 Deliverables

The project is expected to produce an integrated prototype comprising hardware, communication infrastructure, backend services, AI-assisted analysis, user applications, and testing documentation.

No.	Deliverable	Description

1	**IoT Monitoring Prototype**	A sensor-based monitoring setup using the MQ-2 gas and smoke sensor, DHT22 temperature and humidity sensor, PIR occupancy sensor, and Raspberry Pi gateway, subject to equipment availability.
2	**IoT Communication Pipeline**	A communication pipeline using Node-RED and MQTT/MQTTS through HiveMQ to transmit sensor measurements to the backend.
3	**Backend Application**	A Jakarta EE 11 application deployed on WildFly, providing REST APIs and the services required to process measurements and manage incidents.
4	**Database**	A MongoDB data layer for storing sensor measurements, building zones, incident records, alerts, and historical information.
5	**Emergency Detection Module**	A module using configured rules and sensor measurements to identify potential fire and gas leak situations and initiate incident management workflows.
6	**AI Agent**	A LangChain4j-CDI-based agent that retrieves authorized contextual information, assesses potential risks, explains its conclusions, and generates safety recommendations.
7	**Client Progressive Web Application**	A user interface allowing building occupants to consult safety status, active incidents, incident details, and available safety recommendations.
8	**Administrator Progressive Web Application**	A centralized dashboard for monitoring sensors and incidents, managing registered devices and locations, consulting incident history, and performing authorized status updates.
9	**Location-Based Incident Management**	Functionality associating incidents with their originating sensors or affected building zones and supporting targeted notifications when the necessary information is available.
10	**LLMOps and Monitoring**	Operational logging and evaluation of AI requests, tool calls, response times, errors, token usage, and results, subject to implementation scope.
11	**Testing and Validation Report**	Documentation of test scenarios, system behavior, integration results, AI response evaluation, identified limitations, and corrective actions.
12	**Technical Documentation**	Documentation describing the architecture, technologies, configuration, communication flow, APIs, database organization, and deployment procedure.
13	**Final Project Presentation**	A presentation summarizing the problem, objectives, methodology, architecture, implementation, testing results, limitations, and future improvements.

12.1 Expected Final Outcome

At the end of the project, the expected result is a functional prototype demonstrating the main emergency-monitoring workflow:

1. Collect environmental measurements from physical sensors or simulated data sources.
2. Transmit measurements to the backend through the MQTT communication pipeline.
3. Detect potential abnormal situations using configured rules.
4. Retrieve contextual information and perform AI-assisted risk assessment.
5. Create or update an incident and generate relevant recommendations.
6. Display safety information through the client and administrator PWAs.
7. Record operational information to support testing, monitoring, and future improvements.

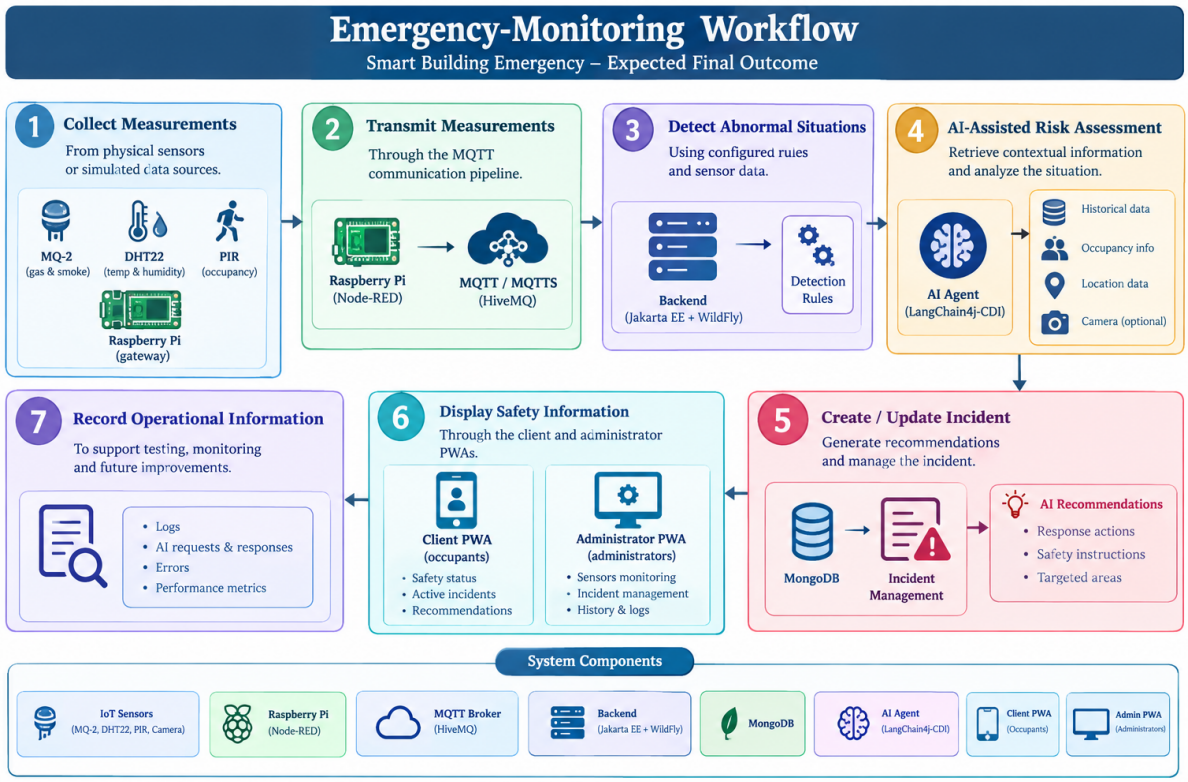


Figure 5: Emergency-monitoring workflow

The project will prioritize the core workflow of sensor data collection, emergency detection, contextual analysis, and incident management. Advanced camera analysis, physical actuator control, and extensive monitoring dashboards will depend on technical feasibility, available resources, and the time remaining.

13 Conclusion

The Smart Building Emergency project aims to design an intelligent platform that improves building safety through connected sensors, real-time monitoring, and AI-assisted emergency analysis. By combining IoT technologies, backend services, data storage, and user-friendly interfaces, the proposed solution will provide a centralized system for monitoring environmental conditions, detecting potential hazards, managing incidents, and supporting emergency response.

The proposed architecture integrates sensors such as the MQ-2, DHT22, and PIR with a Raspberry Pi gateway. Node-RED and MQTT/MQTTS will handle the communication of sensor measurements, while the backend, developed using Jakarta EE and deployed on WildFly, will process the collected data and manage incidents. MongoDB will store sensor measurements and historical records. Furthermore, the integration of an AI agent through LangChain4j-CDI will support contextual risk assessment, explainable analysis, and response recommendations. Client and administrator Progressive Web Applications (PWAs) will provide access to relevant safety information and monitoring functionalities.

The project will follow an iterative development approach based on Extreme Programming (XP), allowing the main components to be implemented, integrated, and tested progressively within the planned seven-week schedule. Particular attention will be given to system integration, reliability, security, and the evaluation of the AI agent's responses.

Finally, this file defines the functional requirements, technical architecture, selected technologies, development methodology, limitations, and expected deliverables of the project. It establishes a clear foundation for implementation and evaluation. The final prototype is expected to demonstrate how IoT and AI can work together to support smarter building safety management, while keeping in mind that automated assessments and AI-generated recommendations cannot replace certified safety equipment or professional emergency procedures.